

Documentación técnica del entrenamiento de los dos modelos. Fase de Machine Learning, julio 2026, semilla global 42, notebooks reproducibles. Todas las métricas son reales, medidas sobre el conjunto de prueba una única vez tras la selección por validación.

B.1 Modelo 1 — Tipo de issue (bug / feature / question)

Dataset. *GitHub Bugs Prediction* (Kaggle), ~150 000 issues reales de GitHub con `title`, `body` y etiqueta (0=bug, 1=feature, 2=question). Fuente: <https://www.kaggle.com/datasets/anmolkumar/git-hub-bugs-prediction>

Preparación. Concatenación `title + body`; limpieza (código → `CODE`, URLs → `URL`, sin HTML, minúsculas); descarte de textos < 10 caracteres; partición estratificada 80/10/10 (semilla 42).

Modelado. TF-IDF (100 000 rasgos, unigramas+bigramas, `min_df=3`, `sublinear_tf`) + clasificador lineal con `class_weight='balanced'`. Selección por F1-macro en validación:

Tabla B.1. Selección del clasificador de tipo por F1-macro en validación.

Candidato	F1-macro (validación)
SVM lineal (LinearSVC)	0,7072
Regresión Logística (seleccionado)	0,7112

Nota. Fuente: Elaboración propia, 2026.

Resultados (test). F1-macro: **0,7097** · Exactitud: 0,778

Tabla B.2. Clasificador de tipo — métricas por clase en el conjunto de prueba.

Clase	Precisión	Recall	F1	Soporte
bug	0,818	0,812	0,815	13 366
feature	0,831	0,783	0,806	13 821
question	0,447	0,588	0,508	2 813

Nota. Fuente: Elaboración propia, 2026.

Análisis. Las clases mayoritarias superan 0,80 de F1. La clase `question` sufre el desbalance (~5:1) y su solapamiento léxico con `bug`; `class_weight='balanced'` prioriza su recall (0,59) sobre su precisión, comportamiento preferible para un sistema de sugerencias.

B.2 Modelo 1 — Severidad (crítica / alta / media / baja)

Dataset. *Eclipse and Mozilla Defect Tracking Dataset* (Lamkanfi, Demeyer, Giger & Goethals, MSR 2013), **214 903 reportes** tras el parseo. Fuente: https://github.com/ansymo/msr2013-bug_dataset

Decisiones metodológicas. (i) Se toma el primer valor del historial de severidad (el del reporte original). (ii) Mapeo Bugzilla → 4 niveles:

Tabla B.3. Mapeo de severidades de Bugzilla a los cuatro niveles del proyecto.

Bugzilla	Proyecto
blocker, critical	crítica
major	alta
normal	media
minor, trivial	baja
enhancement	<i>excluido</i>

Nota. Fuente: Elaboración propia, 2026.

(iii) `normal` es el valor por defecto de Bugzilla y domina ~75 % del corpus; se entrena con él y se reporta su efecto.

Resultados (test). F1-macro: **0,4420** · Exactitud: 0,685 · mismo pipeline TF-IDF + lineal.

Tabla B.4. Clasificador de severidad — métricas por clase en el conjunto de prueba.

Clase	Precisión	Recall	F1	Soporte
crítica	0,506	0,579	0,540	1 968
alta	0,218	0,233	0,225	2 002
media	0,824	0,795	0,810	16 054
baja	0,185	0,203	0,193	1 388

Nota. Fuente: Elaboración propia, 2026.

Análisis. Resultado consistente con la literatura (0,40–0,55 en severidad multiclase; cf. Lamkanfi et al. 2010). El modelo es útil donde más importa: detectar lo **crítico** (F1 0,540, recall 0,579). Las clases `alta` y `baja` son genuinamente ambiguas incluso para anotadores humanos. Por ello la severidad se presenta como sugerencia editable.

(La distribución de clases de ambos clasificadores se ilustra en *fig_distribucion_clases.png*.)

B.3 Modelo 2 — Resumen de commits (diff → mensaje)

Dataset. Corpus limpio de **NNGen** (Liu et al., ASE 2018), derivado del corpus de Jiang, Armaly & McMillan (ASE 2017): pares `diff` → `mensaje` de los 1 000 proyectos Java más populares (~22 100 train / 2 500 valid / 2 500 test). Para el modelo multilenguaje se amplió con **MCMD** (Tao et al., ICSME 2021): 30 000 pares por lenguaje de Python, JavaScript, C++ y C#. Fuente: <https://github.com/Tbabm/nngen>

Arquitectura (desde cero, ~15–35 M parámetros). Codificador: embedding 256 + GRU bidireccional (hidden 256). Decodificador: GRU con **atención de Luong + pointer-generator** (See et al., 2017): $P(w) = p_{\text{gen}} \cdot P_{\text{vocab}}(w) + (1 - p_{\text{gen}}) \cdot \sum \text{atención}$, que permite **copiar identificadores** del diff al mensaje.

Entrenamiento. Teacher forcing, Adam (lr 1e-3), clip 5.0, batch 64, hasta 25 épocas con parada temprana (paciencia 3). Inferencia: beam search (ancho 5) con restricciones *no-repeat bigram* y longitud mínima.

Evolución (test, corpus Java, 2 521 pares).

Tabla B.5. Evolución del resumidor en el corpus Java (conjunto de prueba).

Variante	BLEU-4	ROUGE-L
seq2seq base (greedy)	4,10	—
seq2seq base (beam)	6,68	0,2092
pointer-generator	11,62	0,2927

Nota. Fuente: Elaboración propia, 2026.

El mecanismo de copia mejoró BLEU-4 un +74 % y ROUGE-L un +40 %.

Modelo multilinguaje desplegado (v3). Reentrenado sobre la mezcla de 5 lenguajes; 34,7 M parámetros. **BLEU-4 global: 11,79.**

Tabla B.6. Desempeño del modelo multilinguaje por lenguaje (BLEU-4).

Lenguaje	BLEU-4 (v3)	Antes (solo-Java)
C#	12,72	~0
JavaScript	12,19	~0
Python	11,83	~0
C++	11,82	~0
Java	9,52	11,62
Global	11,79	—

Nota. Fuente: Elaboración propia, 2026.

Trade-off aceptado: -2,1 BLEU en Java a cambio de cobertura real en 4 lenguajes nuevos (~12).

Comparación honesta con la literatura (mismo corpus limpio).

Tabla B.7. Comparación con la literatura sobre el mismo corpus limpio (BLEU-4).

Sistema	BLEU-4
NMT (Liu et al. 2018)	~16,4
NNGen (recuperación, sin red)	~16,4
Este trabajo — seq2seq base	6,68
Este trabajo — pointer-generator	11,62
Este trabajo — recuperación NNGen (réplica)	16,72

Nota. Fuente: Elaboración propia, 2026.

Estrategia híbrida de inferencia. El servicio combina, en orden: (1) heurísticas de ingeniería de software (ChangeScribe), (2) recuperación NNGen ($\text{coseno} \geq 0,5 \wedge \text{BLEU-diff} \geq 0,5$), (3) generativo pointer-generator, (4) respaldo por nombres de archivo. BLEU subestima la calidad percibida (compara n-gramas exactos contra una única referencia en mensajes muy cortos); el sistema es asistencia editable, no generador autónomo.

Limitaciones. Salida en inglés y de un commit individual; diffs muy largos se podan a 100 tokens; corpus mayoritariamente de proyectos Java.